



UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA

Departamento de Ingeniería Informática

Informe laboratorio 2: Paralelismo orientado al objeto

Profesor: Fernando Rannou Fuentes

Integrante: Xavier Muñoz Díaz

Santiago – Chile

Año 2023

Tabla de contenidos

1. Introducción.....	3
2. Desarrollo.....	4
3. Resultados.....	6
4. Conclusiones.....	11

1. Introducción

Las antenas del observatorio ALMA, ubicadas en el Desierto de Atacama, miden señales de radiofrecuencia en longitudes de onda milimétricas y submilimétricas, sin embargo, la señal capturada corresponde a la radiación de esa luz que produce al interactuar con el agua y el polvo que se encuentra alrededor de los objetos por lo que es necesario sintetizar la imagen, esto consiste en transformar esta señal de RF a una señal de intensidad.

Dado que el muestreo es irregular, el objetivo de este laboratorio es diseñar e implementar una aplicación paralela con orientación al objeto usando uC++ mediante un proceso llamado gridding, que se utiliza en síntesis de imágenes interferométricas, por lo que se debe transformar el plano uv no grideado a uno que sí lo sea. También se debe probar el programa en el clúster de la universidad.

Para ello, se comienza a describir la solución con matrices global y otra con matrices locales con el apoyo de los archivos utilizados. Luego se muestran los resultados obtenidos junto con el rendimiento computacional y para finalizar una breve conclusión del laboratorio.

2. Desarrollo

El proceso de gridding consiste en acumular todos los valores de los puntos uv que caen en una celda (en su centro) por lo que se implementan dos soluciones, una con matrices locales y otra con matrices globales.

El programa recibe como parámetros el nombre del archivo de entrada y salida, delta x de la imagen en arco segundos, tamaño de la imagen, el número de chunks de lectura y el número de tareas.

Para poder realizar lo solicitado, se tienen los siguientes archivos:

- `gridding.cc`: Toma los parámetros descritos anteriormente para luego crear una matriz que se utiliza para la acumulación de las visibilidades de las matrices que se encuentran de forma local en cada tarea. Luego, transforma delta x en radianes para poder determinar delta u y delta v. A continuación, se procede a tomar el tiempo para poder inicializar la clase Lectura correspondiente a la corrutina con exclusión mutua y las tareas que utilizan una matriz compartida para guardar y acumular los cálculos de las visibilidades. Una vez hecho los cálculos, se normaliza la matriz real e imaginaria, se termina de tomar el tiempo de ejecución y se escribe en dos archivos el resultado de las visibilidades, uno para la parte real y otro para la parte imaginaria.

Luego se comienza a tomar el tiempo para el algoritmo que posee matrices locales que siguen la misma lógica que el anterior, sin embargo, la diferencia es que cada tarea posee sus propias matrices, es decir, acumulan los valores de las visibilidades en las matrices que poseen. Después que han finalizado las tareas, se procede a acumular todas las matrices de cada tarea para obtener el resultado del gridding, normalizarlas, terminar de calcular el tiempo y escribir en dos archivos de salida.

- `corrutina.cc`: Es de tipo mutex coroutine y se encarga de que una vez activado el main, se lean n chunks por cada tarea que ingrese a esta y les retorna un vector con las líneas leídas. Esto se repite hasta que se quede sin líneas por leer, en cuyo caso retorna un vector de tamaño 0 que sirve como señal para que las tareas se detengan.
- `matriz.cc`: Clase con exclusión mutua que se encarga de definir una matriz compartida para que todas las tareas puedan acceder. Dentro de esta es posible setear y acumular las visibilidades para la matriz real e imaginaria y acumular la matriz correspondiente al peso W. Además, se encarga de normalizar las visibilidades de la matriz real e imaginaria y de escribir los archivos.
- `tarea.cc`: Se construye con punteros a la clase de corrutina y a las matrices globales. Esta ingresa a la corrutina para obtener el vector con las líneas leídas para luego aplicarles split y obtener los valores de las coordenadas u y v, parte real e imaginaria de la visibilidad, el peso W (valor de confianza de la medición, entre

más alto existe menor incertidumbre), la frecuencia de observación y finalmente el canal espectral. Dichos valores se utilizan para transformar las coordenadas u y v a longitud de onda, determina la posición de la matriz que corresponde la visibilidad y lo acumula en la matriz compartida. Las tareas tienen como condición que mientras el vector no esté vacío, repitan el procedimiento.

- `tarea_local.cc`: Similar a la lógica de `tarea.cc`, sin embargo, en el constructor crea matrices para guardar allí las visibilidades.

3. Resultados

Se debe ejecutar la siguiente línea de comando:

```
./gridding -i hltau_completo_uv.csv -o datosgrideados -d 0,003 -N 2048 -c 10 -t 10
```

Ambos algoritmos utilizan los mismos parámetros, en este caso se utilizan los siguientes valores:

- Nombre del archivo de entrada: hltau_completo_uv.csv
- Nombre del archivo de salida: datosgrideados
- Delta x (en arcosegundos): 0,003
- Tamaño de la imagen (matriz): 2048
- Número de chunks: 10
- Número de tareas: 10

Además, para el cálculo del rendimiento computacional se utiliza la siguiente fórmula:

$Speedup = \frac{T_s}{T_p}$, donde T_s es el tiempo secuencial y T_p es el tiempo paralelo.

Para poder determinar el speedup, se realiza un promedio en las 10 ejecuciones del programa como se muestra en la Tabla 1.

	Paralelo		Secuencial	
	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]
	15,1428	17,7790	11,7634	11,6327
	14,1574	16,0732	11,8304	11,7758
	13,5410	15,5630	11,8196	11,3379
	14,2088	15,1710	11,3835	11,7618
	14,2542	16,1398	11,72	11,5768
	12,3317	14,8758	12,1282	11,4664
	14,1980	15,9719	11,4685	11,4639
	14,0577	15,1379	12,3549	11,5899
	14,0779	14,7608	12,729	12,1266
	14,1124	15,1161	11,5423	12,0541
Promedio	14,0082	15,6589	11,8740	11,6786

Tabla 1: Muestra los tiempos de ejecución paralela y secuencial (1 tarea), su promedio y su speedup.
Fuente: Elaboración propia (2023).

	Matrices compartidas [s]	Matrices locales [s]
Número de tareas		
2	11,8403	12,5436
3	11,282	12,0403
4	14,0792	12,6756
5	14,2278	13,1643
6	15,7004	16,3082
7	16,2532	16,9623
8	15,511	17,606
9	16,6502	18,7892

Tabla 2: Ejecución del programa con distinto número de tareas.
Fuente: Elaboración propia (2023).

El algoritmo que posee matrices compartidas, demora en promedio 14,0082 [s] y al utilizar la fórmula de rendimiento computacional resulta 0,847645556 y la imagen resultante se muestra en la Figura 1, la cual es resultado del código de MATLAB mostrado en la Figura 3.

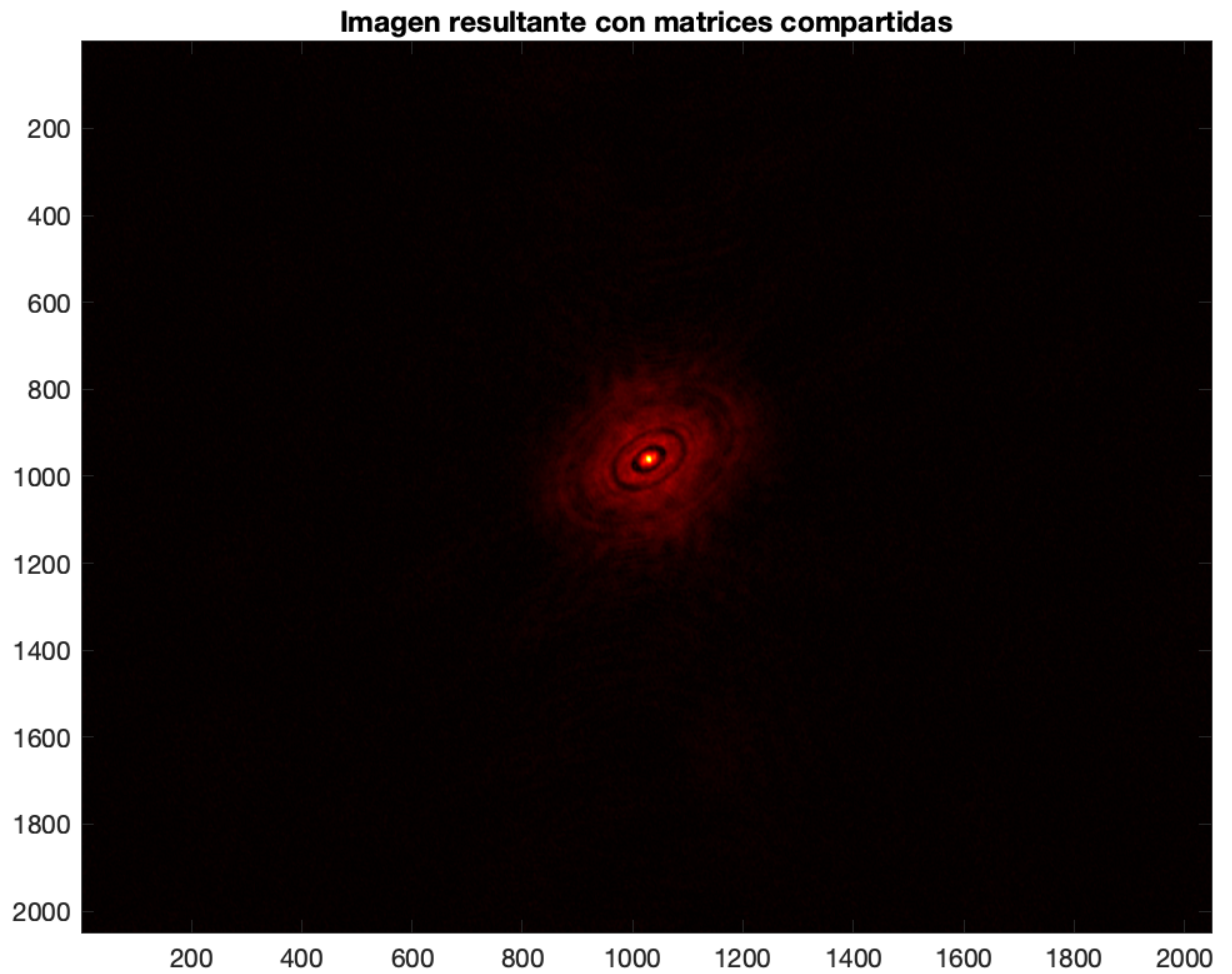


Figura 1: Resultado del grideo de los datos con matrices compartidas.

Fuente: Elaboración propia (2023).

El algoritmo que utiliza matrices locales, demora en promedio 15,6589 [s] y al utilizar la fórmula de rendimiento computacional resulta 0,745814028 y la imagen resultante se muestra en la Figura 2, la cual es resultado del código de MATLAB mostrado en la Figura 3.

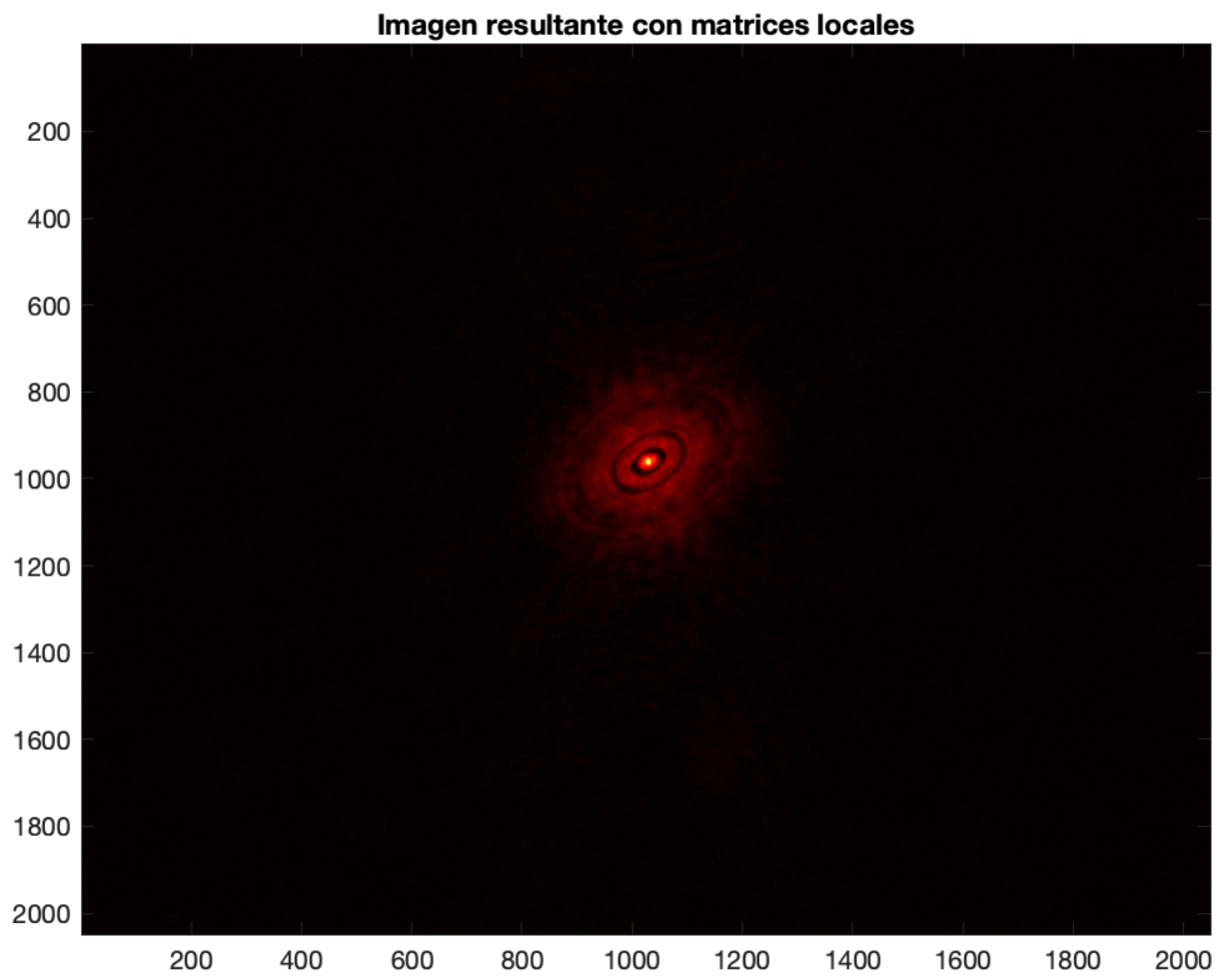


Figura 2: Resultado del grideo de los datos con matrices locales.

Fuente: Elaboración propia (2023).

```

% Abre los archivos de datos grideados con matrices compartidas
f1 = fopen("datosgrideadosr.raw", "rb");
f2 = fopen("datosgrideadosi.raw", "rb");

% Abre los archivos de datos grideados con matrices locales
f3 = fopen("datosgrideadosr_local.raw", "rb");
f4 = fopen("datosgrideadosi_local.raw", "rb");

% Lee los archivos con matrices gloables que poseen valores tipo float de doble precisión
s1 = fread(f1, "double");
s2 = fread(f2, "double");

% Lee los archivos con matrices locales que poseen valores tipo float de doble precisión
s3 = fread(f3, "double");
s4 = fread(f4, "double");

% Toma los vectores creados anteriormente y los transforma en una matriz
re1 = reshape(s1, 2048, 2048);
im1 = reshape(s2, 2048, 2048);
re2 = reshape(s3, 2048, 2048);
im2 = reshape(s4, 2048, 2048);

% Crea vectores complejos
v1 = complex(re1, im1);
v2 = complex(re2, im2);

% Realiza la transformada inversa de Fourier
I = fftshift(iff2(v1));
J = fftshift(iff2(v2));

figure;
imagesc(abs(I));
colormap('hot');
title('Imagen resultante con matrices compartidas');

figure;
imagesc(abs(J));
colormap('hot');
title('Imagen resultante con matrices locales');

% Cierra los archivos
fclose(f1);
fclose(f2);
fclose(f3);
fclose(f4);

```

*Figura 3: Código en MATLAB para ver la imagen resultante.
Fuente: Elaboración propia (2023).*

4. Conclusiones

Dado que fue posible aplicar gridding a los datos de entrada, implementar lo anterior con matrices compartidas y con matrices locales, y poder visualizar la imagen resultante, se cumple dicho objetivo, además, el programa con matrices compartidas en promedio se demora 14,0082 [s] en ejecutar mientras que el que tiene matrices locales demora 15,6589 [s], por lo que el primero es 1,6507 [s] más rápido. Por otro lado, el speed up para el primero resulta en 0,847645556 y para el segundo 0,745814028, esto implica que el programa es aproximadamente un 15.24% y un 25.42% respectivamente más lento en la ejecución de la solución propuesta. Además, se puede observar en la Tabla 2 que hay una tendencia de aumento en el tiempo al ejecutar el programa cuando el número de tareas aumenta. También es posible observar que a simple vista las Figuras 1 y 2 son iguales, dado que solo cambia el método en el que se almacenan inicialmente los valores.

Por otro lado, dado que existieron complicaciones con el cableado de la universidad, no fue posible ejecutar el programa en el clúster de la universidad por lo que el objetivo no fue cumplido.